

COMP9001

Lecture 10

Testing

Presenter: Dr Armin Chitizadeh

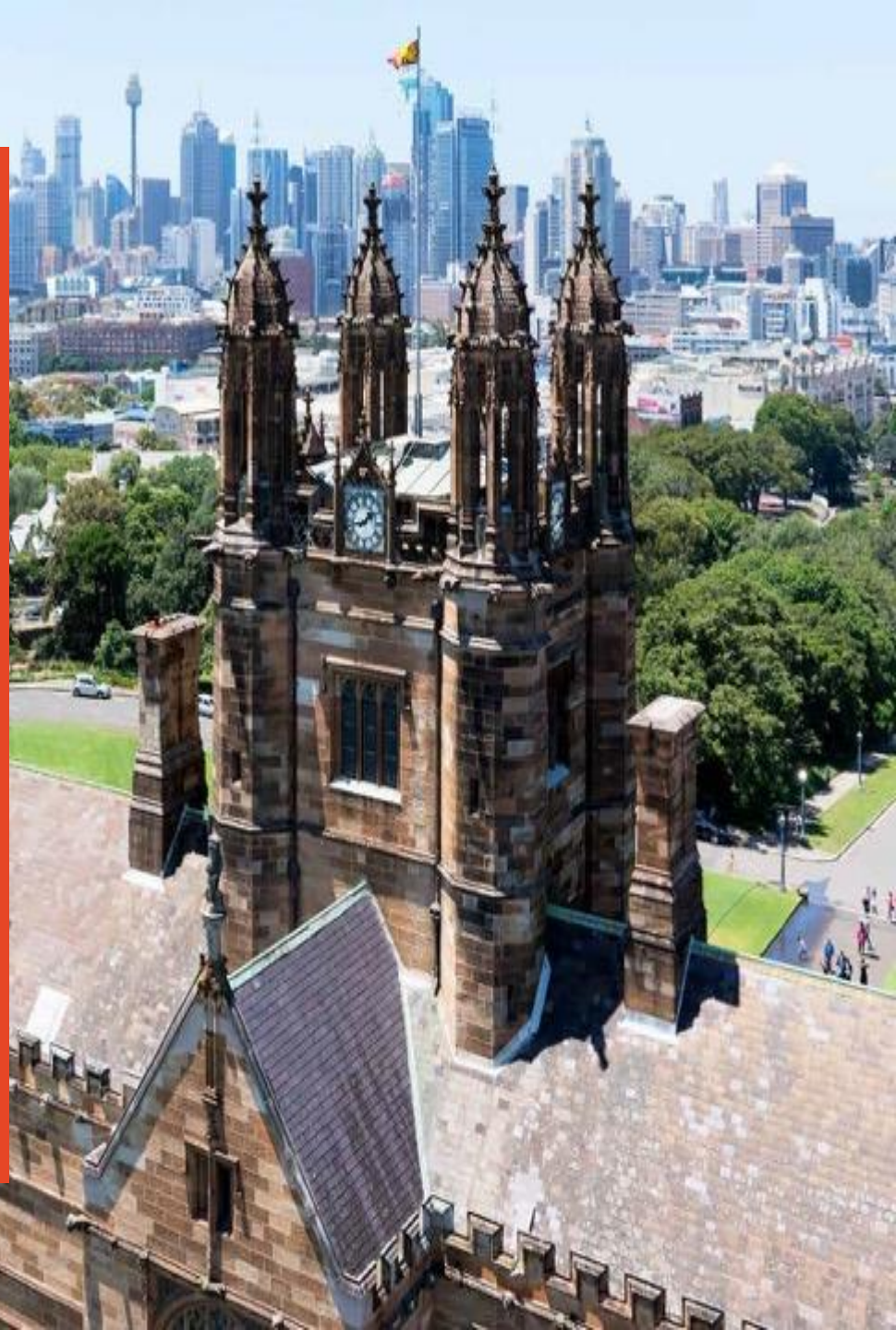
Version 1.0



THE UNIVERSITY OF
SYDNEY

Inspired by Hazem El-Alfy 2024 slides

CRICOS 00026A TEO SA PRV12057



Some Quick Announcements

Assignment2

- If you have not started it, please do soon

Exam

- For the exam, we will provide a formula sheet.
 - We are aiming to upload it next week once it is approved.
- You may bring one A4 sheet of notes, either handwritten or typed.

Final Project: Padlet

- We have uploaded a [3-minute video](#) explaining how to create a Padlet post.
- Your tutor will share the Padlet link with you.
- Please make sure your name is visible to us when posting on Padlet.
- If you have not received the Padlet link by Monday night, please let us know.
- You can find inspiration from:
 - Last year [uploads on Ed](#)
 - Any other resources such as App Store or Google play



No Interview for the second Assignment

- There is no interview for Assignment 2.

Responding to your feedback

- One is announced on ed, two is going to be announced soon

Where are we in the unit?



Why do we need testing?

Ariane 5



https://www.youtube.com/watch?v=PK_yguLapgA

Two distinct but complementary approaches

Testing (dynamic verification)



Photo by [ThisIsEngineering](#) on [Unsplash](#)

Formal Proofs (static verification)

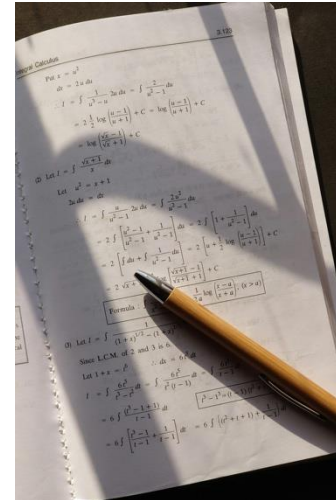


Photo by [Antony Hyson Seltran](#) on [Unsplash](#)

W10: Testing

– Topics

- Types of Tests
- Writing test cases
- `assert` and functions
- `unittest` module (extra)
- `doctest` module (extra)

Life is full of tests!

- Everyone would have undergone some form of testing by now. Examples:
 - National Assessment Program – Literacy and Numeracy (NAPLAN)
 - High School Certificate (HSC)
 - Health examination
- What is a test?
 - *“a procedure intended to establish the quality, performance, or reliability of something, especially before it is taken into widespread use.” – Oxford*
-  Copilot (2026)
 - Testing is a general term that means **checking something to see if it works correctly, meets requirements, or produces expected results**. In computer science, testing refers to: the process of evaluating a software application to **find bugs and ensure it behaves correctly**.

Black Box vs White Box testing

Test Example – Software Testing

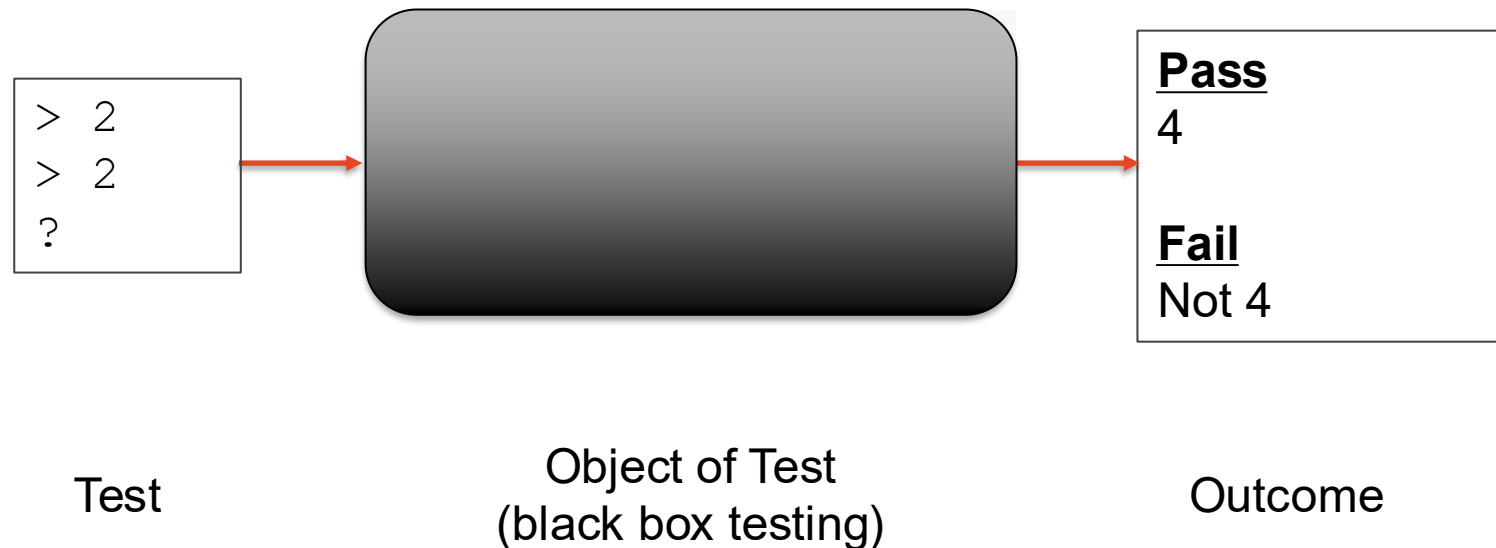
Black Box Testing



Photo by [D.Panyukov](#) on [Unsplash](#)

Test Example – Software Testing

- **Goal:** Software is working as it should be working e.g. produces correct output (response) given inputs.
- **Example:** Write a program that adds two numbers.



Test Example – Software Testing

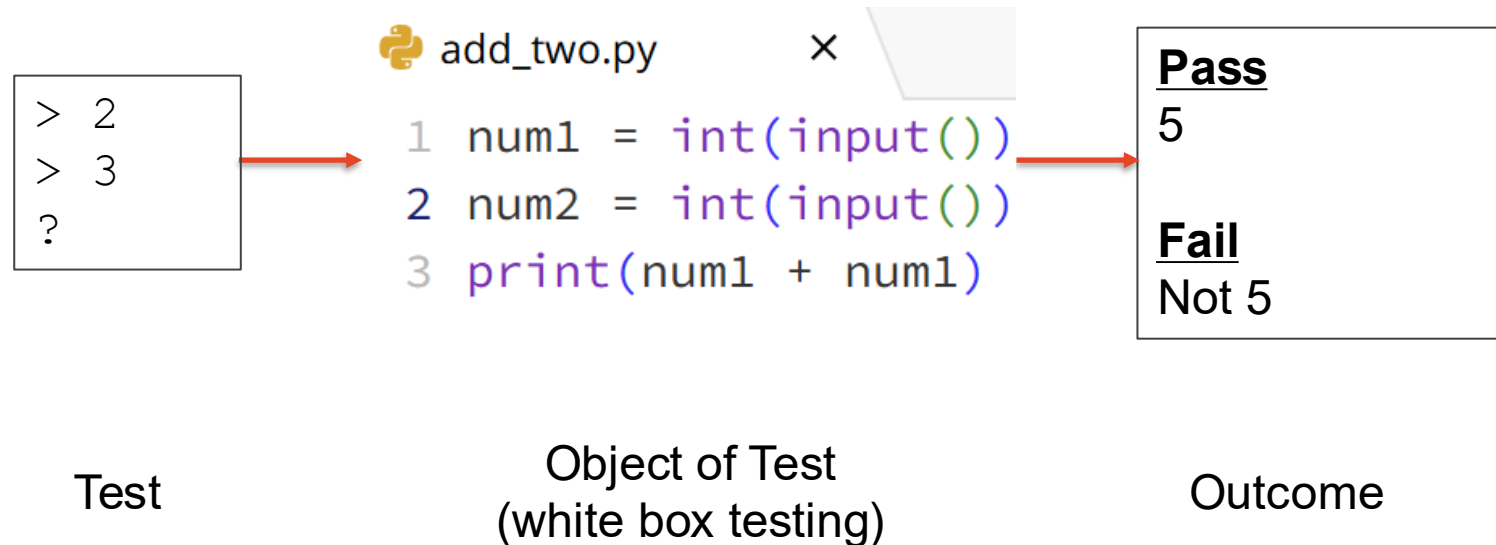
White Box Testing



Photo by Jacob Morch from pexels.com

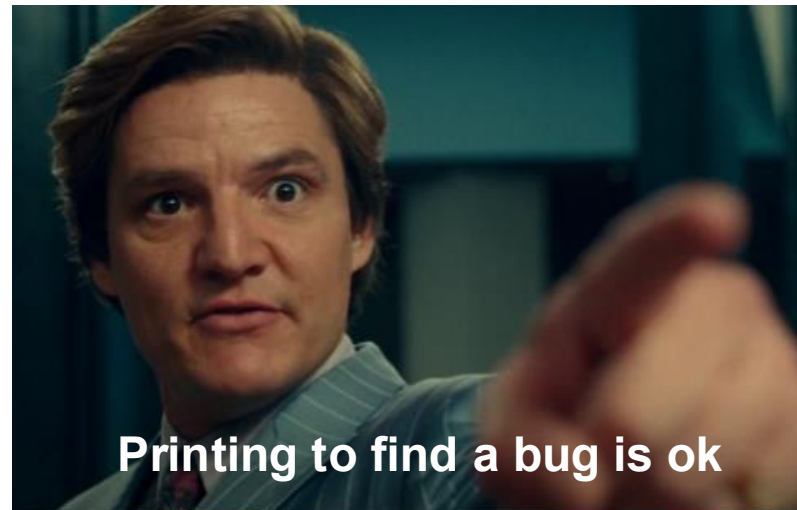
Test Example – Software Testing

- **Goal:** Software is working as it should be working e.g. produces correct output (response) given inputs.
- **Example:** Write a program that adds two numbers.



Debugging Your Way

- Logging states in your code
- Using print statement



Printing to find a bug is ok

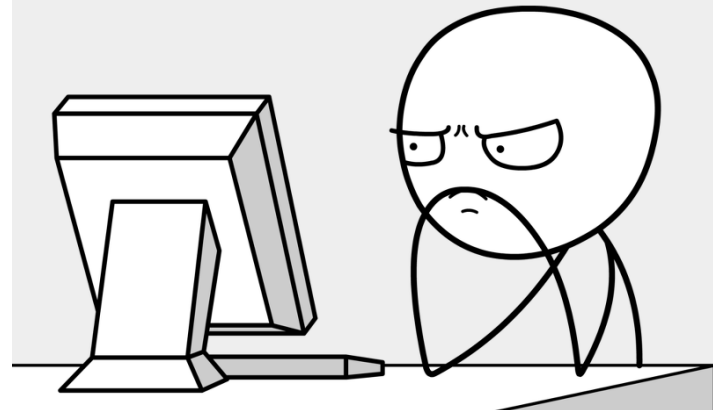


But it can be better

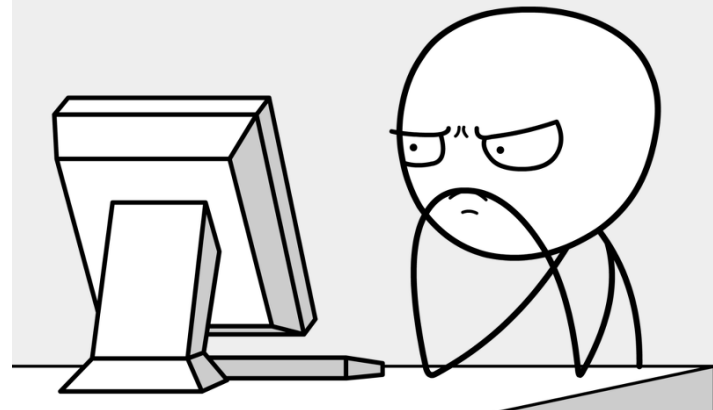
Reality of Testing your Program

- My testing practice survey.
- Testing frustration:
 - I don't know what's wrong with it on Ed. It works on my device.
 - It works for the examples given, why won't the private (hidden) test cases pass?!!
 - Why doesn't it pass test X?!!

Programmers at work:



It doesn't work.... why?



It works.... why?

Test Driven Development (TDD)

- This is a way of software development that requires you to write the tests for a given function *first*, before you even write the function!
- It makes you think exactly what you want your function(s) to do in terms of:
 - What the normal functionality should be (**Positive cases**)
 - What should happen given invalid input (**Negative cases**) (e.g. passing a negative number to a function that expects positive numbers).
 - How it should behave under unusual input (**Edge or corner cases**) (e.g., extreme inputs that test boundaries: very large, very small...).
- Should test whether you are on the right track *throughout* development and not only at the *end*.

Writing tests

- Known as test case design
- Positive case:
 - System response under normal operating parameters (when valid inputs are given).
- Negative case:
 - System response to invalid inputs.
 - Will '**crash**' the program if not handled appropriately.
- Edge case:
 - Extreme cases, e.g. load on a bridge is maximum.
- Corner case:
 - More than one dimension is at an extreme e.g. bridge is maximum load AND hit with winds AND earthquake.

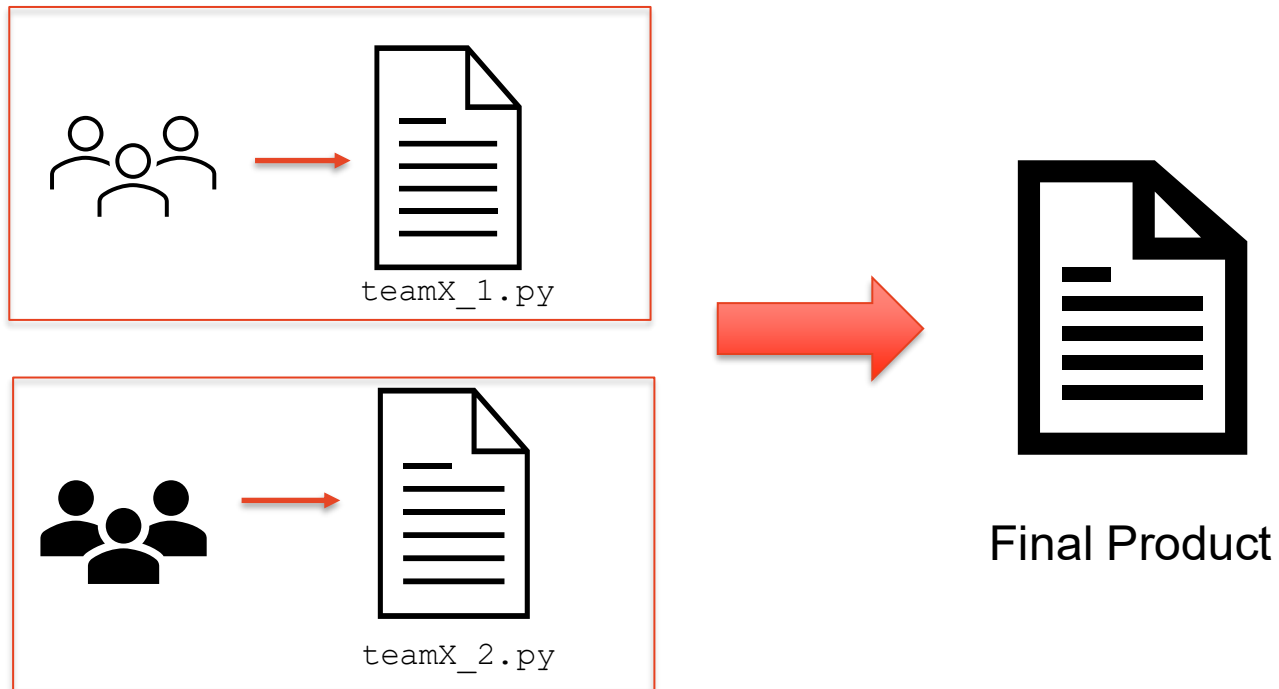


Source: iihs.org

General Testing Process

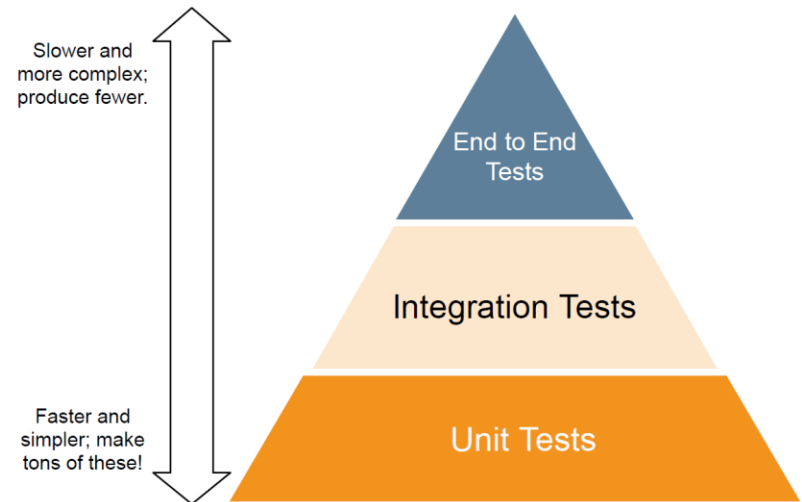
- Determine goal. **[Purpose of the test]**
- Design tests to measure goal. **[Write test cases]**
- Apply tests on test object to measure achievement of goal.
[Implement and run test cases]
- Evaluate test outcome. **[Check outcome and determine action]**

Software



Types of Tests: The Testing Pyramid

- End-to-end Tests
 - Full functionality of system, from start to finish.
- Integration Tests
 - Examines how different components of a system interact with one another.
- Unit Tests
 - Fast and simple tests that target the *smallest* possible components of a system.



Testing – `is_even()` function

- Function to check if a given number (int) is even.
- Returns `True` if number is even. Otherwise, `False`.
- Determine the type of test case and expected output for each of the following inputs. Visually inspect the actual output.
- Check [Ed](#).

Type of case	Input	Expected Output
	<code>is_even(5)</code>	
	<code>is_even(2)</code>	
	<code>is_even(5.5)</code>	
	<code>is_even("HELLO")</code>	
	<code>is_even([])</code>	
	<code>is_even(-999999999999999999999999)</code>	

assert statement

- Syntax:

```
assert <condition>, <msg>
```

- Meaning:

- If <condition> is False, raise AssertionError with the message: <msg>.
- if <condition> is True, then nothing happens.
- AssertionError
 - An exception that is raised by the assert statement fails.
 - <https://docs.python.org/3/library/exceptions.html#AssertionError>

- Go to Ed Lesson

- Review the codes in demo.py. What do you notice about both code blocks?
- Predict the output of the program. Run the program to check your answer.

Testing using the assert statement

- Review the codes in `integer.py` on Ed.
- Is the function behaving according to specification?
- Determine the output of the program if the function:
 - Satisfies the requirements?
 - Does not satisfy the requirements?
- Run the program to check your answer.

```
1 def is_even(num):
2     '''
3     Function to check if a given number is even.
4     num: int, number to check if it is even.
5     result: bool, returns True if it is even. Otherwise, False.
6     '''
7
8     # Type Checking
9     is_integer = isinstance(num, int)
10
11     if not is_integer:
12         return False
13
14     # Check if it is even or odd.
15     result = (num % 2 != 0)
16     return result
17
18 assert is_even(5) == False, "False, 5 is odd"
19 assert is_even(2) == True, "False, 2 is even"
20 assert is_even(5.5) == False, "False, num is a float."
21 assert is_even("HELLO") == False, "False, num is a string."
22 assert is_even([]) == False, "False, num is an empty list."
```


Manual Testing: The problem

- Every time you fix something, you should re-run all your *previous* tests
- Regression testing: Running ALL your tests again whenever you make a significant change to your code to make sure you haven't broken code that used to work.
- Solution: **Automated Testing**
 - Allows testing to be repeated as many times as needed until the problem is fixed.
 - Reduces human error in executing the test.
 - Report of testing results.
- Ideally, test codes should be in a **separate file** to minimise the risk of modifying the test object during test development. This file is called the **test driver**.

(Revisit) Basics of Programming

- All procedural programming languages have loops, conditionals and functions. These basic constructs can be used to solve *any* problem.
- The syntax may be different for other languages, but the principles remain the same.
- Why should we use functions when we can write codes without functions?
 - Code re-usability
 - Reduce errors
 - Independent testing – **unit testing**

```
1 num = int(input('Integer: '))
2
3 # Check Conditions
4 is_even = num % 2 == 0
5 is_within_range = num >= 20 and num <= 200
6 is_negative = num < 0
7
8 # Test conditions
9 test1 = is_even and is_within_range
10 test2 = not(is_even) and is_negative
11 test_result = test1 or test2
12
13 # Display test_result
14 if test_result:
15     out = f"{num} passes the test!"
16 else:
17     out = f"{num} fails the test :("
18 print(out)
19 # End program
```

(Revisit) Good Practices: Functions and Modules

- A module contains functions that are available to be used by other programs.
- A client is a program that uses a function in a module.
 - `import` statement at the top of the program.
 - Syntax: `import <module_name>`
 - Qualify function calls to the module using dot operator
 - Syntax: `<module_name>.<function_name>`
- The module must be in the same directory as the client.

Module

```
1 '''
2 integer.py
3 '''
4 def is_even(num: int)-> bool:
5     '''
6     Returns True if num is even. Otherwise, False.
7     '''
8     result = num % 2 == 0
9     return result
10
11 def within_range(num: int)->bool:
12     '''
13     Returns True if num is within range. Otherwise, False.
14     '''
15     result = num >= 20 and num <= 200
16     return result
17
18 def is_negative(num: int)-> bool:
19     '''
20     Returns True if num is negative. Otherwise, False.
21     '''
22     result = num < 0
23     return result
```

Client

```
driver.py
1 import integer
2
3 integer.main()
```

Good Practices: Test Driver

- Import the module that contains the function to be tested.
- Define a separate function for each test case.
- Call the functions to run the test cases accordingly.

```
1 from integer import is_even
2
3 def test01():
4     num = 5
5     actual = is_even(num)
6     expected = False
7     assert actual == expected, f"{expected}, {num} is even?"
8     print("Testcase 01 pass.")
```

- Can you tell which test cases pass and which fail?
- Did you run all the tests?
- Go to [Ed Lesson](#) and run `test_driver.py`

Practice: Testing – search function

- Write `assert` statements for each test case inside the `main()` body. Is this a good testing practice?
- Write test functions to implement each test case. Run the tests. Does the program tell you whether the test cases pass or fail?
- Go to [Ed Lesson](#) and run `animal_hunt.py`

Summary of Testing

- Determine the purpose of testing.
- Write test cases – positive, negative and edge cases.
- For each test case: description, input, output, code.
- Implement the test cases.
 - Import the module that contains the function to be test.
 - Define a function for each test case.
 - Assert statements to check the **expected output** with the **actual output**
 - Save or display the result of testing.
 - Call the functions to run the test cases accordingly.
 - `try ... except` maybe needed for negative cases.

It's Break Time!

5 min



THE UNIVERSITY OF
SYDNEY



Assert statement is good



But it can be better

Extra: Testing Automation Tools

- Assert statements **stops** execution if a test fails.
- The remaining test cases will **not** be executed and you have no way of knowing which other test cases will fail.
- Many unit testing tools available:
 - **unittest module** (<https://docs.python.org/3/library/unittest.html>)
 - **nose module** (<https://nose.readthedocs.io/en/latest/>)
 - **pytest module** (<https://docs.pytest.org/en/6.2.x/#>)
 - **doctest module** (<https://docs.python.org/3/library/doctest.html>)

Testing Terminology

- **Test case:** Individual unit of testing. Checks for a specific response to a particular set of inputs.
- **Test suite:** A collection of test cases, test suites, or both. It is used to aggregate tests that should be executed together.
- **Test runner:** Component which orchestrates the execution of tests and provides the outcome to the user.
- Python documentation: <https://docs.python.org/3/library/unittest.html>

unittest module

- A unit testing framework that supports test automation.
- List of assert methods to check for and report failures:

unittest Method	Checks that
<code>assertEqual(a, b)</code>	<code>a == b</code>
<code>assertNotEqual(a, b)</code>	<code>a != b</code>
<code>assertGreater(a, b)</code>	<code>a > b</code>
<code>assertGreaterEqual(a, b)</code>	<code>a >= b</code>
<code>assertAlmostEqual(a, b)</code>	<code>round(a-b, 7) == 0</code>
<code>assertNotAlmostEqual(a, b)</code>	<code>round(a-b, 7) != 0</code>

- Python documentation:
 - <https://docs.python.org/3/library/unittest.html#unittest.TestCase.debug>
 - <https://docs.python.org/3/library/unittest.html#unittest.TestCase.assertIsInstance>

unittest basics

1. Import `unittest` module and the module that is subject to the test.

2. Create a class that inherits from `TestCase` class:

```
class <classname>(unittest.TestCase):
```

3. Create instance methods for each test

- Method name conventionally starts with letters `test`, e.g. `test01`, to inform the test runner which methods represent tests.
 - Method body contains `unittest` `assert` statements
- [Do Part 1 in Ed Lesson](#)

```
1 import unittest
2 from integer import is_even
3
4 class TestInteger(unittest.TestCase):
5     # Test case 01. Postive case.
6     def test_01(self):
7         num = 5
8         actual = is_even(num)
9         expected = False
10        self.assertEqual(actual, expected)
11
12    # Test case 02. Postive case.
13    def test02(self):
14        num = 2
15        actual = is_even(num)
16        expected = True
17        self.assertEqual(actual, expected)
18
19
20    def test03(self):
21        num = 5.2
22        actual = is_even(num)
23        expected = True
24        self.assertEqual(actual, expected)
25
26    def test04(self):
27        num = "HELLO"
28        actual = is_even(num)
29        expected = False
30        self.assertEqual(actual, expected)
31
32    def test05(self):
33        num = []
34        actual = is_even(num)
35        expected = False
36        self.assertEqual(actual, expected)
37
38 if __name__ == "__main__":
39     unittest.main()
```

unittest basics

- Ways to run tests from terminal:

1. `python3 <filename>`
2. `python3 -m unittest <modulenames>`

- What's the difference?

- Code to run tests and display test results to terminal is contained in `unittest.main()`.
- If Lines 38-39 are omitted in `test_driver.py`:
 - Approach 1: Will not work. `unittest.main()` is not called.
 - Approach 2: Will work. Run `test_driver.py` from the `unittest` module with test modules as its arguments.
`$ python3 -m unittest test_driver.py`

- Go to [Ed Lesson and do Part 2.](#)

Recap: docstrings

- Documenting is necessary for communication to another reader about what the code does.
- When placed in double ' ' ' right at the start of the class/function, it becomes a runtime attribute accessible via `__doc__`. Accessible by other programmers:
`module.<function/class>.__doc__`
- Docstrings are expected to provide:
 - Short description of the function
 - Parameters and return values
 - **Example usage** (`>>>`)
 - Exceptions raised
- A complex function requires example usage that could be placed in the docstring.

```
doc.py
1 # This is a comment, not a string.
2 ''' This is the docstring for this module. '''
3 import math
4 import doctest
5
6 def my_sqrt(x):
7     ''' This is the docstring for func1.
8     >>> my_sqrt(2)
9     1.4142135623730951
10
11     >>> my_sqrt(4)
12     2
13
14     '''
15     result = math.sqrt(x)
16     return result
17
18 def main():
19     print(my_sqrt.__doc__)
20     print(my_sqrt(2))
21     print(my_sqrt(4))
22
23 if __name__ == '__main__':
24     main()
25     doctest.testmod()
26 # End program
```

Writing Testable Documents

- `doctest` module - searches for pieces of text that look like interactive Python sessions and then executes those sessions to verify that they work as shown.
- Why should you perform doctest?
 - To check that a module's docstrings are up-to-date by verifying that all interactive examples still work as documented.
 - To perform regression testing by verifying that interactive examples from a test file or a test object work as expected.
- Python documentation: <https://docs.python.org/3/library/doctest.html>

Implementing test: `doctest.testmod()`

```
doc.py
1 # This is a comment, not a string.
2 ''' This is the docstring for this module. '''
3 import math
4 import doctest
5
6 def my_sqrt(x):
7     ''' This is the docstring for func1.
8     >>> my_sqrt(2)
9     1.4142135623730951
10
11     >>> my_sqrt(4)
12     2
13
14     '''
15     result = math.sqrt(x)
16     return result
17
18 def main():
19     print(my_sqrt.__doc__)
20     print(my_sqrt(2))
21     print(my_sqrt(4))
22
23 if __name__ == '__main__':
24     main()
25     doctest.testmod()
26 # End program
```

1. Test cases added into docstring.
 - Expected output to follow final `>>>`
2. Import `doctest` module.
3. Call `testmod()` function.
4. Analyse test results.
 - Nothing will be displayed if all test cases pass.

Ed Lesson on docstring

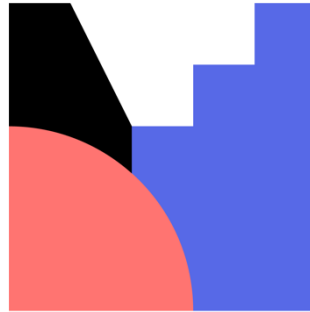
Supplementary: Software and Accidents

- Ariane 5
 - Turning 90 degrees in air and self-destructing.
 - https://youtu.be/gp_D8r-2hwk
- NHS IT blunder
 - Wrong prescriptions for 300, 000 heart patients
 - <https://www.dailymail.co.uk/health/article-3585149/Up-300-000-heart-patients-given-wrong-drugs-advice-major-NHS-blunder.html>
- Uber Self-Driving Car
 - Software did not recognize pedestrian. SUV did not need to stop!
 - Driver charge for being ‘visually distracted’. Pedestrian killed.
 - <https://www.npr.org/2019/11/07/777438412/feds-say-self-driving-uber-suv-did-not-recognize-jaywalking-pedestrian-in-fatal->

Reading This Week

- **Chapter 6.13.** Liang, Y. (2022). Introduction to Python Programming and Data Structures, Global Edition. Pearson Education Limited.
- **Chapter 8. Testing and debugging.** Guttag, J. V. (2021). Introduction to Computation and Programming Using Python: With Application to Computational Modeling and Understanding Data. Mit Press.
- **Chapter 1, 2, 4. Sale.** (2014). Testing Python: Applying Unit Testing, TDD, BDD and Acceptance Testing. Wiley.
- Recommended Reading:
 - **Chapter 6.** Kernighan, & Pike, R. (1999). The practice of programming. Addison-Wesley.

It's Menti Time!



THE UNIVERSITY OF
SYDNEY

<https://www.menti.com/alx19tnxrrhy>

Discovery Activity: The Karate Kid!



You are being trained by Mr Miyagi (Miyagi Sensei) for the upcoming All-Valley Tournament! In order to ensure you are prepared, Mr Miyagi develops a training program for you and will only let you enter the tournament if you are ready.

- (a) You must be able to perform 20 kicks
- (b) You should be able to balance a pot on your head for at least 30 seconds
- (b) When he calls your name you should respond "Yes Sensei!"

Use unit tests to prove to Mr. Miyagi that you are ready for the All-Valley Tournament! [Ed Link Here](#)

Fully Anonymous feedback

Fully anonymous feedback form for COMP9001 students – Fill out form

